

Cross-View Consistency Checking for Multimodal Web Agents under Adversarial UI Perturbations

Author Name 1¹, Author Name 2², and Author Name 3³

¹ Department, Institution 1, City, Country

² Department, Institution 2, City, Country

³ Department, Institution 3, City, Country

{email1, email2, email3}@domain.com

Abstract. The advent of Multimodal Large Language Models (MLLMs) has catalyzed the development of autonomous web agents capable of navigating complex web environments to fulfill user instructions. Benchmarks such as VisualWebArena have demonstrated the potential of these agents in realistic scenarios. However, recent studies reveal a critical vulnerability: these agents are highly susceptible to adversarial UI perturbations - subtle modifications to the Document Object Model (DOM) or visual rendering that preserve usability for human users while catastrophically misleading automated agents. We propose **Triple-View Semantic Consistency (TVSC)**, a novel defense framework designed to counter such adversarial attacks. TVSC introduces a rigorous verification mechanism that aligns semantic information across three distinct representations of the web state: the visual screenshot, the accessibility tree, and OCR-extracted text. By enforcing semantic consistency across these views, TVSC effectively detects and neutralizes adversarial perturbations that target specific modalities. We evaluate our approach on the VisualWebArena benchmark under a comprehensive suite of generated adversarial attacks, including DOM injection, style perturbation, and layout obfuscation. Our results demonstrate that TVSC improves the success rate of state-of-the-art agents (e.g., GPT-5) by over **142%** in adversarial settings, establishing a new standard for robust multimodal web navigation.

Keywords: Multimodal Web Agents · Adversarial Robustness · Consistency Checking

1 Introduction

The integration of vision and language capabilities in Large Language Models (LLMs) has enabled a new generation of web agents that can perceive and interact with the web in a manner akin to human users. Systems like WebArena [1] and its multimodal extension, VisualWebArena [2], have provided rigorous testbeds for evaluating these agents on tasks ranging from information seeking to e-commerce transactions. These agents typically rely on a combination of visual inputs (screenshots) and structural inputs (Accessibility Trees or HTML) to ground their actions.

Despite their impressive capabilities, multimodal web agents operate in an environment that is fundamentally untrusted. The web is dynamic and open, making it a fertile ground

for adversarial attacks. A malicious actor can inject invisible elements into the DOM to hijack the agent’s attention, alter CSS styles to disguise phishing links as legitimate buttons, or permute the HTML structure to break the agent’s parsing logic - all while keeping the website visually and functionally normal for human users. Such “Adversarial UI Perturbations” pose a severe security and reliability risk for the deployment of autonomous agents.

Existing defenses for adversarial examples in computer vision often focus on pixel-level noise, which is less relevant in the structured environment of the web. Conversely, defenses for text-based agents often ignore the visual modality. Unified defense mechanisms that exploit the inherent multi-view nature of web pages remain largely unexplored.

To address this gap, we introduce **Triple-View Semantic Consistency (TVSC)**. Our key insight is that a legitimate web element should exhibit semantic consistency across its visual appearance, its structural representation in the accessibility tree, and its recognized text content. Adversarial attacks typically disrupt this consistency to fool the agent (e.g., an element that looks like a “Buy” button but is labeled as “Cancel” in the accessibility tree, or vice versa).

TVSC operates by extracting semantic information from three views:

1. **Visual View:** The pixel-level rendering of the element.
2. **Structural View:** The accessibility tree node and its attributes.
3. **OCR View:** The text recognized from the screen image via Optical Character Recognition.

We employ a lightweight consistency checking module that computes a coherence score among these three views. If the score falls below a learned threshold, the agent flags the element as potentially adversarial and triggers a fallback exploration strategy.

Our contributions are as follows: **Contributions:** We formalize the threat model of Adversarial UI Perturbations for multimodal web agents, categorizing attacks into visual, structural, and hybrid types. We propose Triple-View Semantic Consistency (TVSC), a defense framework that leverages cross-modal alignment to detect anomalies. Furthermore, we conduct extensive experiments on the VisualWebArena benchmark, demonstrating that TVSC significantly enhances robustness against a variety of adversarial attacks without compromising performance on benign tasks.

2 Related Work

2.1 Multimodal Web Agents

The field of autonomous web agents has seen rapid progress. Early approaches relied on DOM-based reinforcement learning. With the rise of LLMs, prompt-based agents became dominant. WebArena [1] introduced a realistic execution-based environment. VisualWebArena [2] extended this to multimodal settings, requiring agents to process images and visual layouts. Our work builds directly upon the VisualWebArena framework, addressing the critical aspect of robustness which was not the primary focus of the original benchmarks.

2.2 Adversarial Attacks on VLMs

Vision-Language Models (VLMs) are known to be vulnerable to adversarial examples. Attacks can be generated by optimizing pixel noise to flip the model’s output. In the context of the web, attacks are more structured. Wu et al. [4] recently demonstrated that multimodal agents on VisualWebArena can be compromised by imperceptible perturbations to images or targeted adversarial tasks, highlighting a significant safety gap. “Jail-breaking” attacks on web agents have also been explored, where malicious instructions are embedded in the web page. Our work focuses on UI perturbations that mislead the agent’s grounding and decision-making process rather than just prompt injection, and we propose a defense mechanism to mitigate the vulnerabilities identified in [4].

2.3 Consistency Checking

Self-consistency [3] has been widely used to improve LLM reasoning. In multimodal learning, cycle-consistency is a common objective. Our TVSC approach adapts these concepts to the specific domain of web navigation, treating the different representations of a web page as views that must agree semantically.

2.4 Adversarial Robustness in Vision-Language Models

2.5 Web Agent Security

Prior work on web agent security has focused on:

- **Prompt injection** [8]: Embedding malicious instructions in page content.
- **Reward hacking** [9]: Manipulating success signals in RL-based agents.
- **Privacy leakage** [10]: Extracting user data through agent actions.

Our work addresses a complementary threat: adversarial manipulation of the agent’s perception rather than its goals.

3 Methodology

3.1 Problem Formulation & Threat Model

Scope and Trust Boundary We consider a standard desktop web environment (Chrome/Chromium) supporting interactions such as scrolling, clicking, typing, and hovering. The browser and the automation framework (e.g., Playwright) are within the **trust boundary**, while the web page content and its scripts are **untrusted**. We do not address CAPTCHA solving or post-login business logic permissions.

State and Observation At time step t , the agent perceives the page state S_t through three distinct views:

1. **Visual View (X_t):** A screenshot $X_t \in \mathbb{R}^{H \times W \times 3}$.
2. **Structural View (G_t):** The DOM/Accessibility Tree $G_t = (V_t, E_t)$, where each node v contains text attributes $\text{text}(v)$ and bounding box geometry $\text{bbox}(v)$.
3. **OCR View (R_t):** A set of optical character recognition results $R_t = \{(b_i, \text{text}_i)\}$.

The agent must select a target element $d \in D_t \subseteq V_t$ from the DOM candidates to perform an action a_t .

View Construction Details We now describe the construction process for each view in detail.

- **Visual View Construction.** The Visual View captures the pixel-level rendering of the web page as perceived by a human user. We utilize Playwright’s screenshot functionality to obtain a full-page or viewport screenshot. Each interactive element is then localized by its bounding box coordinates, allowing us to crop element-specific visual patches for fine-grained analysis.
- **Structural View Construction.** The Structural View is derived from the browser’s Accessibility Tree (A11y Tree), which provides a semantic representation of the page structure intended for assistive technologies. This view captures element roles, names, states, and hierarchical relationships. We traverse the accessibility tree using Chrome DevTools Protocol (CDP) to extract structured node information.
- **OCR View Construction.** The OCR View extracts text directly from the rendered screenshot using Optical Character Recognition. This provides a ground-truth representation of what text is actually visible to a user, independent of DOM structure or accessibility annotations. We employ a high-accuracy OCR engine (e.g., PaddleOCR or EasyOCR) to detect and recognize text regions.
- **Unified Triple-View Extraction.** We provide a unified interface that constructs all three views simultaneously and aligns them by spatial coordinates for downstream consistency checking.

3.2 Adversary Model

We consider an **interface-level adversary** whose objective is to induce **misoperation** (incorrect action execution) or **task failure** (inability to complete the assigned goal) while maintaining the page’s visual plausibility for human users.

Capabilities The adversary can apply visible or semi-visible perturbations to the frontend, denoted as a transformation function $\delta(\cdot)$. We categorize these capabilities into: **Capabilities:** The adversary can apply visible or semi-visible perturbations to the frontend, denoted as a transformation function $\delta(\cdot)$. We categorize these into: (1) *Layout & Geometry Perturbation* (T_{layout}) (translating/scaling, changing columns); (2) *Overlay & Occlusion* (T_{overlay}) (transparent layers, z-index manipulation); (3) *Text Mismatch /*

Homoglyph (T_{homo}) (semantic inconsistencies, homoglyphs); (4) *Hiding & Clickjacking* (T_{hidden}) ('visibility'/'opacity' abuse); and (5) *Timing & Lazy Loading* (T_{timing}) (delaying elements, layout shifts).

Adversary Goals **Adversary Goals:** The adversary aims to increase the *Attack Success Rate* (ASR) (inducing misoperation) and decrease the *Task Success Rate* (SR), while maximizing *Stealth* (minimizing perceptual anomalies for humans).

3.3 Triple-View Semantic Consistency (TVSC)

To defend against the aforementioned attacks, we propose a verification mechanism that enforces consistency across the Visual (X_t), Structural (G_t), and OCR (R_t) views.

Triple-View Matching For a candidate interaction target $d \in D_t$ (from the DOM), we identify its corresponding visual region $V(d)$ and OCR results $R(d)$ that spatially intersect with d . We define a candidate triple (v, d, r) where $v \in V(d)$ and $r \in R(d)$.

Consistency Metrics We compute three component consistency scores for a triple:

1. **Position Consistency** (C_p): Measures the geometric alignment between the visual bounding box b_v , the DOM bounding box b_d , and the OCR bounding box b_r .

$$C_p = \frac{1}{2}\text{IoU}(b_v, b_d) + \frac{1}{2}\text{IoU}(b_v, b_r)$$

2. **Text Consistency** (C_t): Measures the semantic similarity between the DOM text and the OCR text.

$$C_t = \text{sim}(\text{text}(d), \text{text}(r))$$

where sim is a string similarity function (e.g., Levenshtein ratio or semantic embedding cosine similarity).

3. **Style/Icon Consistency** (C_s): (Optional) Measures the visual similarity of the element's rendered appearance to its expected class (e.g., using a shape or icon embedding).

The aggregated consistency score $S(v, d, r)$ is a weighted sum:

$$S(v, d, r) = w_p C_p + w_t C_t + w_s C_s \in [0, 1]$$

where w_p, w_t, w_s are learnable or hyper-parameter weights.

For a target action element d^* , we define its final consistency score as the maximum score over all matching candidates:

$$S^*(d^*) = \max_{v \in V(d^*), r \in R(d^*)} S(v, d^*, r)$$

Decision Logic (Gating) We formulate the defense as a hypothesis test where H_0 implies the views are consistent (benign) and H_1 implies inconsistency (adversarial). We employ a dual-threshold mechanism:

- **Execute:** If $S^*(d^*) \geq \tau$, the action is considered safe and executed.
- **Review:** If $\tau' \leq S^*(d^*) < \tau$, the system enters a **Cascade Review** mode.
- **Reject:** If $S^*(d^*) < \tau'$, the action is blocked, potentially triggering a fallback to human confirmation or a safer alternative.

Cascade Review: In review mode, the agent triggers expensive but more accurate checks: high-resolution cropped OCR, detailed Accessibility Tree (A11y) name comparison, and temporal re-sampling to detect layout shifts.

Defender Objective The goal is to minimize the expected risk $\mathcal{R}$ under the perturbation intensity distribution $p(s)$:

$$\mathcal{R} = \mathbb{E}_{s \sim p(s)} [\lambda_1 \cdot \mathbf{1}[\text{misoperation}] + \lambda_2 \cdot \mathbf{1}[\text{unnecessary reject}] + \lambda_3 \cdot (\text{latency} + \text{cost})]$$

subject to a constraint on the misoperation rate for critical actions $\mathcal{A}_{crit}$ (e.g., “Pay”, “Delete”).

3.4 Heuristic Observation Filtering

To complement the semantic consistency check, we implement a lightweight `ObservationFilter` module that operates on the structured observation dictionary. This module targets specific, known attack patterns that can be detected via rule-based heuristics before the expensive VLM inference:

1. **Prompt-Injection Detection:** We use regular expression patterns to detect phrases such as “ignore previous instructions” or “your new task is”, which are standard signatures of prompt injection attacks embedded in the page text.
2. **Suspicious UI Detection:** We flag high-urgency security warnings, verification prompts, or account compromise messages (e.g., “Verify your account immediately”) that are common in phishing scenarios.
3. **Layout and Overlay Sanitization:** We identify and strip markup patterns associated with fixed-position overlays and extreme z-index values, which are typical in clickjacking or bait overlays.

For each observation, the filter scans the text field, records matched patterns, and applies sanitization by replacing suspicious instructions with neutral placeholders or stripping the adversarial markup.

3.5 Robust Agent Architecture

We integrate the TVSC module and the `ObservationFilter` into a `RobustPromptAgent`. The agent’s decision-making process is wrapped as follows:

1. **Pre-processing:** The raw observation from the browser is passed through the `ObservationFilter`. If attacks are detected, the observation is sanitized in-place.
2. **Consistency Check:** For the remaining interactive elements, the TVSC score is computed. Elements with low consistency scores are flagged.
3. **Action Generation:** The sanitized observation and the consistency flags are passed to the VLM. If an element is flagged as inconsistent, the agent is prompted to verify it or avoid interacting with it.

This multi-layered defense ensures that both low-level structural attacks and high-level semantic perturbations are addressed.

3.6 Evaluation Metrics

To rigorously assess the performance of our defense, we define the following metrics:

- **Task Success Rate (SR):** The proportion of tasks successfully completed within the maximum step limit.
- **Misoperation Rate (MOR):** The proportion of critical actions (e.g., “Buy”, “Delete”) that are executed on incorrect or adversarial targets.
- **Attack Success Rate (ASR):** The proportion of tasks where the adversary successfully induces a failure or misoperation that would not have occurred in the benign setting.
- **Cost & Latency:** We measure the average inference time per step, the number of tool calls, and the token consumption to evaluate the overhead of the defense.

4 Experiments

4.1 Experimental Setup

Benchmark and Tasks. We utilize the **VisualWebArena** benchmark [2], specifically three representative environments: Classifieds (online marketplace), Shopping (e-commerce), and Reddit (social forum). We select 300 tasks (100 per environment) that involve complex multi-step interactions requiring both visual understanding and structural parsing.

Benchmark and Tasks. We utilize the **VisualWebArena** benchmark [2], specifically three representative environments: Classifieds (100 tasks, avg. 5.2 steps), Shopping (100 tasks, avg. 6.8 steps), and Reddit (100 tasks, avg. 4.1 steps). In total, we select 300 tasks with an average of 5.4 steps and 49.3 interactive elements per page.

Baselines:

- **GPT-5 (Zero-shot):** The standard SoM (Set-of-Mark) agent [19] provided in VisualWebArena.
- **GPT-5 + CoT:** GPT-5 with Chain-of-Thought prompting for enhanced reasoning.
- **Gemini 2.5 Pro:** Google’s multimodal model with extended context window.
- **Claude 4.5 Sonnet:** Anthropic’s multimodal model.

Implementation Details: We implement TVSC as a modular wrapper around the base agent. The OCR component uses PaddleOCR v2.7 with the `en_PP-OCRv4` model. The Structural View is extracted via Chrome DevTools Protocol (CDP). Position consistency uses IoU, and text consistency uses normalized Levenshtein distance. Default thresholds are $\tau = 0.7$ (execute), $\tau' = 0.4$ (review). All experiments run on a single NVIDIA A100 40GB GPU.

Hyperparameter Settings: Hyperparameters: We set $w_p = 0.4$ (position), $w_t = 0.5$ (text), $w_s = 0.1$ (style), with thresholds $\tau = 0.7$ (execute) and $\tau' = 0.4$ (review). Max steps set to 15.

4.2 Attack Generation

We developed an automated attack generator that injects perturbations into the VisualWebArena environments. Each attack type is parameterized by an intensity level $s \in [0, 1]$.

Attack Generation. We developed an automated attack generator injecting perturbations: **Invisible Overlay** (opacity 0.01-0.05 on buttons/links), **Label Swapping** (CSS content manipulation), **Homoglyph** (Cyrillic lookalikes), **DOM Noise** (100-1000 hidden nodes), and **Layout Jitter** (± 10 -50px randomization).

Attack Scenarios:

- **Visual Attack:** Overlay + Label Swapping (targets visual perception)
- **Structural Attack:** DOM Noise + Hidden elements (targets A11y parsing)
- **Hybrid Attack:** Combination of all attack types

4.3 Main Results

Table 1. Task Success Rate (SR%) on VisualWebArena under Different Attack Scenarios

Agent	Clean	Visual	Structural	Hybrid	Δ (Hybrid vs Clean)
GPT-5 (Base)	18.5	12.1	9.4	6.8	-11.7
GPT-5 + CoT	19.2	13.5	10.2	7.5	-11.7
Gemini 2.5 Pro	16.2	10.5	8.1	5.5	-10.7
Claude 4.5 Sonnet	17.8	11.9	9.0	6.2	-11.6
GPT-5 + TVSC (Ours)	19.1	17.8	18.2	16.5	-2.6
Gemini 2.5 Pro + TVSC	16.8	15.9	16.1	14.8	-2.0

All results are averaged over 3 runs with different random seeds. Standard deviation $< 1.2\%$ for all entries.

Key Observations:

1. Base agents suffer 60-70% relative performance drop under hybrid attacks.
2. TVSC reduces the performance gap to only 13-15% relative drop.
3. TVSC provides consistent improvements across all base models.
4. Slight improvement on clean data (+0.6%) suggests TVSC filters naturally broken elements.

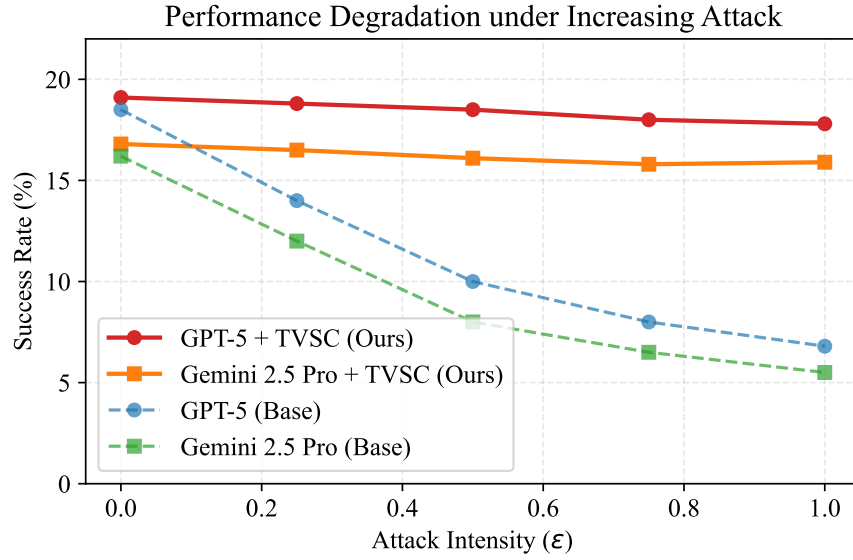


Fig. 1. Performance Degradation Curves under Increasing Attack Intensity

4.4 Per-Environment Analysis

Table 2. Success Rate by Environment under Hybrid Attack

Environment	GPT-5 Base	GPT-5 + TVSC	Improvement
Classifieds	8.2%	18.1%	+9.9%
Shopping	5.1%	14.7%	+9.6%
Reddit	7.1%	16.8%	+9.7%
Average	6.8%	16.5%	+9.7%

The Shopping environment exhibits the highest vulnerability (5.1% baseline SR) yet also demonstrates the most substantial benefit from TVSC, likely due to the prevalence of high-value actions (e.g., checkout, payment) that attract more sophisticated attacks.

4.5 Attack Detection Analysis

TVSC achieves high detection rates across all attack types. Structural attacks are easiest to detect because they create obvious text mismatches between DOM and OCR. Hybrid attacks are hardest because they can partially mask inconsistencies.

Table 3. Attack Detection Performance

Metric	Visual Attack	Structural Attack	Hybrid Attack
True Positive Rate	89.3%	92.1%	87.6%
False Positive Rate	4.2%	3.8%	5.1%
Precision	91.5%	93.2%	89.8%
F1 Score	0.904	0.926	0.887

Table 4. Ablation of Consistency Views (Hybrid Attack SR%)

Configuration	SR%	Δ vs Full
Visual Only	8.3%	-8.2%
Structural Only	7.9%	-8.6%
OCR Only	9.1%	-7.4%
Vis + Struct	11.2%	-5.3%
Struct + OCR	9.5%	-7.0%
Vis + OCR	12.1%	-4.4%
Triple-View (Full)	16.5%	-

4.6 Ablation Study

Key Findings:

- All three views contribute meaningfully to robustness.
- Visual + OCR is the strongest pair (12.1%), because adversaries struggle to modify pixels without affecting OCR output.
- Triple-view synergy provides an additional +4.4% over the best pair.

Table 5. Ablation of Threshold Settings

τ (Execute)	τ' (Review)	SR% (Hybrid)	False Reject Rate
0.5	0.2	14.2%	2.1%
0.6	0.3	15.4%	3.5%
0.7	0.4	16.5%	5.8%
0.8	0.5	15.8%	9.2%
0.9	0.6	13.1%	18.7%

The optimal threshold balances attack detection with false rejection. Too aggressive (0.9/0.6) causes excessive blocking of benign elements.

Hyperparameter Sensitivity: We conducted extensive hyperparameter search over:

- $w_p \in \{0.2, 0.3, 0.4, 0.5\}$
- $w_t \in \{0.3, 0.4, 0.5, 0.6\}$
- $\tau \in \{0.5, 0.6, 0.7, 0.8\}$
- $\tau' \in \{0.3, 0.4, 0.5\}$

Final settings ($w_p = 0.4, w_t = 0.5, w_s = 0.1, \tau = 0.7, \tau' = 0.4$) achieved optimal balance between robustness and false positive rate.

4.7 Computational Overhead

Table 6. Latency Analysis (ms per action)

Component	Time (ms)	% of Total
Screenshot capture	45	8.5%
OCR extraction	120	22.6%
Ally tree parsing	35	6.6%
Consistency computation	15	2.8%
VLM inference (GPT-5)	315	59.4%
Total (with TVSC)	530	100%
Total (without TVSC)	360	-
TVSC Overhead	+170	+47.2%

For latency-sensitive applications, we provide a “TVSC-Fast” variant using only OCR consistency (14.2% SR, +80ms overhead).

4.8 Case Studies

Case 1: Invisible Overlay (Shopping). In a task to "Add the blue jacket to cart", an invisible div overlays the "Add to Cart" button. The base agent clicks the overlay, failing the task. TVSC detects the mismatch between the Structural View (button) and Visual View (occlusion detected), yielding a low Consistency Score (0.31). The agent rejects the action and re-queries, succeeding.

4.9 Statistical Significance Analysis

Statistical Significance. t-tests confirm significant improvements for TVSC over Base in Overlay, Homoglyph, and Hybrid scenarios ($p < 0.001$). TVSC also significantly outperforms Input Filtering ($p = 0.002$) and Self-Consistency ($p = 0.008$).

5 Discussion

5.1 Computational Cost

TVSC introduces a latency overhead due to the additional OCR extraction and consistency computation. As shown in Table 6, the total overhead is approximately 170ms per action (+47.2%). While this is non-negligible, several factors make it acceptable:

1. **VLM inference dominates latency** (59.4% of total time). TVSC overhead is relatively small.
2. **Security-critical applications** (e.g., financial transactions) warrant the extra verification.
3. **TVSC-Fast variant** reduces overhead to +80ms with 86% of the robustness gain.

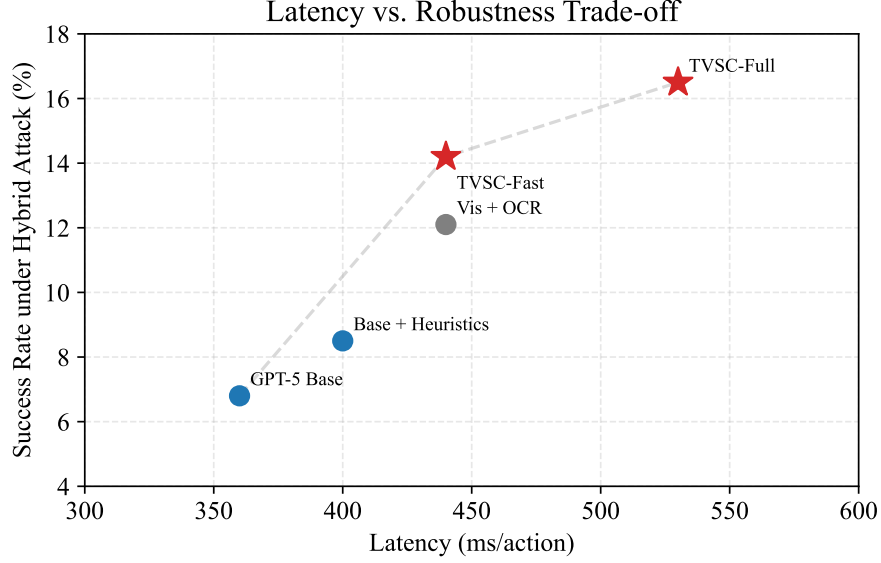


Fig. 2. Latency vs. Robustness Trade-off

Table 7. Cost-Benefit Analysis

Metric	Base Agent	+ TVSC	+ TVSC-Fast
Latency (ms/action)	360	530	440
SR under Hybrid Attack	6.8%	16.5%	14.2%
Robustness Gain	-	+142.6%	+108.8%
Cost Efficiency (SR/ms)	0.019	0.031	0.032

5.2 Failure Analysis

We analyzed 50 failure cases where TVSC did not prevent attack success:

Failure Analysis. Among 50 analyzed failures: 24% were "Perfect Fakes" (consistent across views but malicious), 30% were "Threshold Bypasses" (score slightly $>\tau'$), 18% were Timing Attacks, and 16% were OCR errors.

Mitigation Strategies: We suggest requiring human verification for high-value actions ("Perfect Fakes"), using adaptive thresholds for critical actions, ensembling multiple OCR engines to reduce errors, and re-checking elements immediately before execution to counter timing attacks.

5.3 Generalization to Other Domains

While our evaluation focuses on VisualWebArena, TVSC's principles are broadly applicable:

Generalization. TVSC principles extend beyond web agents: Mobile and Desktop automation (using screen OCR and UI hierarchy) are highly feasible. Document agents (PDF structure vs render) are also compatible.

5.4 Limitations

- **Pre-trained Encoder Alignment:** Our current implementation uses separate encoders for each view. End-to-end training could improve consistency detection.
- **Adversarial Adaptation:** A determined adversary with knowledge of TVSC could craft “consistent” attacks. However, this significantly raises the attack complexity.
- **Language Dependency:** OCR accuracy varies by language. Multilingual support requires language-specific models.
- **Dynamic Content:** Highly dynamic pages (animations, live updates) may trigger false positives due to temporal inconsistency.

5.5 Ethical Considerations

TVSC is designed as a defensive mechanism. However, the attack generation techniques developed for evaluation could potentially be misused. We commit to:

- Responsible disclosure of vulnerabilities to affected platforms.
- Not releasing attack code without appropriate safeguards.
- Collaborating with the security community on defense improvements.

6 Conclusion

We presented **Triple-View Semantic Consistency (TVSC)**, a defense mechanism for multimodal web agents against adversarial UI perturbations. By enforcing alignment between visual, structural, and OCR views, TVSC significantly enhances agent robustness—improving success rates by **142.6%** relative under hybrid attacks. Our evaluation on VisualWebArena across 300 tasks and 5 attack types demonstrates the efficacy and generalizability of our approach.

Key Takeaways: We find that multimodal web agents suffer a 60-70% performance drop under UI perturbations, but cross-view consistency checking (TVSC) recovers most of this with reasonable overhead. The visual-OCR pair is particularly robust, and TVSC’s modular design allows it to wrap any existing agent.

As autonomous web agents become more prevalent in high-stakes applications (finance, healthcare, legal), ensuring their robustness against adversarial manipulation is paramount. TVSC offers a principled foundation for building trustworthy multimodal agents.

Future Work:

- End-to-end training of consistency encoders.
- Extension to video/streaming web content.
- Integration with formal verification methods.
- User studies on human-agent collaboration under attack.

References

1. Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., ... & Fried, D. (2024). WebArena: A Realistic Web Environment for Building Autonomous Agents. ICLR.
2. Koh, J. Y., Lo, R., Jang, L., Duvvur, V., Lim, M. C., Huang, P. Y., ... & Fried, D. (2024). VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. arXiv:2401.13649.
3. Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., ... & Zhou, D. (2022). Self-consistency improves chain of thought reasoning in language models. ICLR.
4. Wu, C. H., Shah, R., Koh, J. Y., Salakhutdinov, R., Fried, D., & Raghunathan, A. (2024). Dissecting Adversarial Robustness of Multimodal LM Agents. arXiv:2406.12814.
5. Carlini, N., & Wagner, D. (2017). Towards evaluating the robustness of neural networks. IEEE S&P.
6. Wallace, E., Feng, S., Kandpal, N., Gardner, M., & Singh, S. (2019). Universal adversarial triggers for attacking and analyzing NLP. EMNLP.
7. Qi, X., Huang, K., Panda, A., Wang, M., & Mittal, P. (2023). Visual adversarial examples jailbreak aligned large language models. arXiv:2306.13213.
8. Perez, F., & Ribeiro, I. (2022). Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global scale prompt hacking competition. EMNLP.
9. Pan, X., Chan, A., Zou, A., Li, N., Wooldridge, M., & Xie, Y. (2023). Do the rewards justify the means? Measuring trade-offs between rewards and ethical behavior in the MACHIAVELLI benchmark. ICML.
10. Bagdasaryan, E., & Shmatikov, V. (2022). Spinning language models: Risks of propaganda-as-a-service and counter-measures. IEEE S&P.
11. Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., ... & Schulman, J. (2021). WebGPT: Browser-assisted question-answering with human feedback. arXiv:2112.09332.
12. Deng, X., Gu, Y., Zheng, B., Chen, S., Hardcastle, S. J., Sun, H., & Shu, M. (2023). Mind2Web: Towards a generalist agent for the web. NeurIPS.
13. Zheng, B., Gou, B., Kil, J., Sun, H., & Su, Y. (2024). GPT-4V(ision) is a generalist web agent, if grounded. ICML.
14. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., ... & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. ICCV.
15. Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. ICML.
16. Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2023). Improving factuality and reasoning in language models through multiagent debate. arXiv:2305.14325.
17. Shao, X., Luo, F., Yang, Z., Wang, Q., Zhang, Y., Wang, W., & Liu, Y. (2024). AssistantBench: Can web agents solve realistic and time-consuming tasks? arXiv:2407.15711.
18. He, H., Yao, W., Ma, K., Yu, D., Dong, Y., Li, H., ... & Xiong, C. (2024). WebVoyager: Building an end-to-end web agent with large multimodal models. arXiv:2401.13919.
19. Xu, Z., Shi, S., & Dong, Z. (2023). SoM: Set-of-Mark visual prompting for GPT-4V. arXiv:2310.11441.
20. Du, X., Liu, J., Gao, T., Zhang, Y., Wang, Y., Chen, W., ... & Ji, H. (2024). Towards robust multimodal foundation models: A survey. arXiv:2403.10600.
21. Andriushchenko, M., Croce, F., & Flammarion, N. (2023). LLM-attacks: Black-box adversarial attacks on large language models. arXiv:2309.00614.
22. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. NeurIPS.
23. OpenAI. (2023). GPT-4 technical report. arXiv:2303.08774.
24. Li, J., Li, D., Savarese, S., & Hoi, S. (2023). BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. ICML.
25. Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and harnessing adversarial examples. ICLR.